Ders 1 Genişletilmiş Notu: Dersin İşleyişi, İşletim Sistemine Giriş, Kesmeler, Depolama Hiyerarşisi ve Çok İşlemcili Sistemler

Ders 1 Genişletilmiş Notu: Dersin İşleyişi, İşletim Sistemine Giriş, Kesmeler, Depolama Hiyerarşisi ve Çok İşlemcili Sistemler

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yapısı ve Kaynaklar
- 2. Dönemin Yol Haritası
- 3. Bilgisayar Sistemi ve İşletim Sisteminin Tanımı
- 4. Sistemin Türüne Göre İşletim Sisteminden Beklentiler
- 5. Kontrol Programı, Çekirdek ve Sistem Programları
- 6. Önyükleme (Bootstrap)
- 7. Aygıt Denetleyicileri ve Kesmeler
- 8. Kesmenin Zaman Çizelgesi ve G/Ç İçin Sistem Çağrısı
- 9. Depolama Yapısı ve Hiyerarşisi
- 10. Önbelleğe Alma ve Tutarlılık Sorunu
- 11. Doğrudan Bellek Erişimi (DMA) ve von Neumann / Harvard Mimarileri
- 12. Çok İşlemcili Sistemler
- 13. Kümelenmiş Sistemler
- 14. Çoklu Programlama ve Zaman Paylaşımı
- Örnek Sorular
- Dersin Toparlaması

1. Dersin Yapısı ve Kaynaklar

İşletim Sistemleri dersi iki parçadan oluşur. Bu şubenin Pazartesi günleri iki saatlik teori dersi vardır. Salı günleri ise bütün şubelerin ortak uygulama (laboratuvar) saatleri vardır. Uygulamalar her hafta yapılmaz. Laboratuvarda bazı uygulamaları yapabilmek için teoride belli bir noktaya gelmiş olmak gerekir. Hoca bunun tahminen dördüncü hafta civarı olacağını söyledi.

Laboratuvarda dört ya da beş uygulama planlanıyor. Bir önceki yıl dört uygulama yapılmıştı, bu yıl bir tane daha eklenmesi düşünülüyor. Uygulamaların konuları şunlardır:

- Kurulacak bir Linux ortamında kabuk (shell) komutları ve kabuk programlama (shell programming).
- Süreçler arası iletişim (interprocess communication).
- Önyükleyici (bootloader) üzerine çalışma.
- Mümkün olursa çekirdek modülü (kernel module) yazma.

Hoca bu uygulamaların derste konuşulan şeylerin karşılığı olduğunu vurguladı. Derste konuşulan şey kuram, laboratuvarda yapılan şey gerçek hayattır.

> **Hocanın vurgusu:** Laboratuvar uygulamalarından da sorumlusunuz. Orada yaptığınız ya da yapmış olmanız gereken şeyler değerlendirmelerde önünüze çıkacak.

Ders için üç temel kitap vardır:

- Silberschatz, Galvin, Gagne, *Operating System Concepts*.
- Tanenbaum, *Modern Operating Systems*.
- Stallings, *Operating Systems*.

Ders, Silberschatz'ın kitabının 9. baskısının slaytları üzerinden yürüyecek. Bu slaytlar internette bulunabilir. Hoca 10. baskının da çıktığını, ama derslerin 9. baskı üzerinden götürüleceğini söyledi. Slaytlar yorumlanırken Tanenbaum ve Stallings'in kitaplarındaki bilgiler de araya katılacak. Çünkü bu kitaplar büyük ölçüde ortak olsa da ayrıldıkları ve farklı bilgi içerdikleri noktalar da vardır.

> **Ders sonrası not:** Hoca Stallings'in kitabının adından emin olamadı. Kitabın tam adı büyük olasılıkla *Operating Systems: Internals and Design Principles*'tır. Altyazıdaki "Şilber Şats", "Shivershots", "Shower Shots" Silberschatz demektir.

2. Dönemin Yol Haritası

Hoca, bu ilk derste bir giriş yapılacağını ve dönem boyunca nelerin konuşulacağını şöyle özetledi:

- **Bilgisayar sisteminin organizasyonu ve mimarisi:** Bir işletim sisteminden söz edebilmek için önce bilgisayar sisteminin nasıl kurulduğunu bilmek gerekir. Bunların bir kısmı biliniyor olabilir, ama bir kısmının hatırlatılması ya da doğru biçimde yeniden öğrenilmesi gerekir.
- **Veri yapıları:** Belki şimdiye kadar öğrenilmiş ama hiçbir yerde kullanılmamış veri yapıları, işletim sisteminin içinde kullanılır. İşletim sistemi bilgisayar mühendisliğinde çok çalışılmış, oturmuş ve kararlı bir konudur. Bağlı liste (linked list), ağaç (tree) ve özetleme (hash) yapılarının hepsi bir yerde kullanılır.
- **İşletim sisteminin görevleri:** Bir yazılıma işletim sistemi diyebilmek için yerine getirmesi gereken işlevler vardır. Üç temel görev konuşulacak: süreç yönetimi (process management), bellek yönetimi (memory management) ve depolama yönetimi (storage management).
- **Korunma ve güvenlik (protection and security):** Bir bilgisayar sisteminde çalışan bir işletim sisteminden söz ediliyorsa, buradaki mekanizmalar da konuşulmalıdır.
- **Özel sistemler:** Her gün görülen sistemler dışında, veri merkezlerinin sistem odalarında görülebilen kümeler (cluster) ve bıçak sunucular (blade server) gibi sistemlerin temel kavramları.

3. Bilgisayar Sistemi ve İşletim Sisteminin Tanımı

İşletim sistemini tanımlarken farklı açılardan bakıp farklı tanımlar yapmak mümkündür. Önce bilgisayar sisteminin bileşenlerine bakmak gerekir. Bir bilgisayar sistemi dört temel bileşenden oluşur:

- **Donanım (hardware):** En temel haliyle işlemci, bellek ve G/Ç aygıtları (I/O devices). Klavye, fare, ikincil bellek olan sabit disk, ekran ve yazıcı birer G/Ç aygıtıdır. Çevre birimlerinin çoğu G/Ç aygıtıdır.
- **İşletim sistemi:** Uygulamaların kaynak kullanımıyla ilgili isteklerini bir düzene sokan, kaynakların düzgün ve etkin kullanılmasını sağlayan kontrol programı.
- **Uygulama programları:** Sistemde bir amaca yönelik çalışan her şey bir uygulamadır. Yazı yazmak için metin düzenleyici (text editor) açılır, oyun oynamak için oyun başlatılır.
- **Kullanıcılar:** Kullanıcı sıradan bir insan olabileceği gibi başka bir bilgisayar sistemi, bir yazılım ya da bir donanım da olabilir. Karşı tarafta her zaman bir insan olmasını beklememek gerekir.

3.1 Kullanıcı bakışı

Kullanıcı açısından bakıldığında işletim sistemi, kullanıcı uygulamalarının çalıştırılmasını sağlayan bir yapıdır. Bunu yaparken problemleri de çözmesi gerekir. Burada problem çözmekten kasıt, uygulamanın yaptığı hesaplama değildir. Bir sistemi birden fazla kişi ya da uygulama kullanır. İşletim sistemi bunların birbirini bozmayacağı, düzgün davranacağı ve sistemden en etkin biçimde yararlanacağı bir düzen kurar. Ayrıca sistemin kolay kullanılabilir olmasını sağlar. Yani işletim sistemi biraz da amaçlarıyla tanımlanır.

3.2 Sistem bakışı: kaynak yöneticisi

İşletim sistemi açısından bakıldığında elimizde bir bilgisayar mimarisi, yani donanım vardır. Bu donanımın kullanıcılar ve kullanıcılara ait uygulamalar tarafından kullanılması gerekir. İşletim sisteminin görevi, bu kaynakları kaynak talep eden uygulamalar arasında paylaştırmaktır.

> **Hocanın vurgusu:** İşletim sistemi çok temelde bir kaynak yönetim sistemidir (resource allocator).

3.3 İyi bir işletim sistemi az çalışır

Hoca bu noktada önemli bir ilkeyi vurguladı: İyi ve etkin bir işletim sistemi, bilgisayar sisteminin çalışmasını en az ölçüde kesintiye uğratır. İşletim sisteminin fazla çalışması, asıl iş yapması gereken uygulamaların daha az CPU zamanı alması demektir. İşletim sisteminin bir kaynağı gereksiz yere ya da gereğinden fazla kullanması istenmez. Etkinlik biraz da buradan gelir.

4. Sistemin Türüne Göre İşletim Sisteminden Beklentiler

İşletim sisteminin ne yaptığı ya da hangi amaca hizmet ettiği, bir ölçüde bakış açısına, bir ölçüde de bilgisayar sisteminin ne için kurulduğuna bağlıdır.

Kişisel bilgisayar: Bir bilgisayar ve onu kullanan bir kişi vardır. Bu kişi sistemin bütün kaynaklarına hakimdir ve onları tek başına kullanır. Bu durumda istenen şey iyi başarım (performance), kararlılık ve kullanım kolaylığıdır (convenience). Kaynakların tam kullanımı (utilization) ise dert edilmez. Hoca bunu şöyle örnekledi: Kimse “bu işlemciye 400 dolar verdim, parasını hak etmiyor, onu daha çok çalıştırayım” diye düşünmez. Ekran kartlarına yüksek para verenlerin hız aşırtma (overclock) merakı bir yana, sıradan bir kullanıcı her dakika CPU zamanını ve belleği sonuna kadar kullanmayı amaçlamaz.

Paylaşımlı sistemler (sunucu, mainframe): Birden fazla kullanıcının olduğu ortamlarda kullanıcılar arasında bir denge kurulması ve kaynakların kullanıcıların isteklerine göre dağıtılması beklenir. Bu, herkese aynı CPU zamanının, aynı belleğin ve aynı depolamanın verilmesi demek değildir. Çok kullanıcılı (multi-user) ortamlarda kullanıcıların çeşitli seviyeleri vardır. Bu seviyeler servis anlaşmalarına ya da üyelik anlaşmalarına göre belirlenir. Hocanın deyişiyle “money talks”: Herkes ödediği ya da sistemde tanımlandığı kadar pay alır.

Tablet ve cep telefonu: Bu cihazlar pil üzerinde çalışır. Pil de bir kaynaktır ve iyi kullanılması gerekir. Gereksiz bir durum varsa ekran kapatılır. Gerekirse CPU düşük güç (low power) çalışma düzenine geçer. Böylece başarım ölçütleri ve sistemin çalışma biçimi tamamen değişir.

Sonuç olarak işletim sisteminin işi genel anlamda kaynak yönetimidir. Ama bu kaynakların ne olduğuna ve ortamın ne olduğuna göre iş kendi içinde farklılaşır.

5. Kontrol Programı, Çekirdek ve Sistem Programları

5.1 Kontrol programı

İşletim sistemi kaynak tahsisi (resource allocation) yapar. Bir diğer yönüyle de bir **kontrol programıdır (control program)**. Kendisi de aslında bir uygulamadır ve kullanıcı uygulamalarını çalıştırır. Kullanıcı uygulamaları çalışırken onları denetler. Hataların oluşmasını ve bilgisayarın uygunsuz kullanımını (improper use of the computer) engeller.

Uygunsuz kullanımdan kasıt şudur: Sistem açılınca önce donanım kendini ilklendirir, sonra kontrol işletim sistemine geçer. İşletim sistemi de sistemin ölçütü neyse (çok kullanıcılı mı, tek kullanıcılı mı, kaynak kullanımı mı, etkileşimli olmak mı) ona göre uygulamaları çalıştırır. Bir uygulamanın yetkisi olmayan bir şeyi çalıştırmaya kalkması, başka bir uygulamayı bozmaya çalışması ya da bilgisayar sisteminin kontrolünü ele geçirmeye çalışması uygunsuz kullanımdır. Kontrol programı bunların önüne geçer.

5.2 Tek bir tanım yoktur

Her şeyi kapsayan bir işletim sistemi tanımı yoktur. Mainframe'den akıllı telefona, oradan da mikrodenetleyicilere (microcontroller) kadar inen bir yelpazede her sistemin isterleri farklıdır. Bu yüzden global bir tanım yapmak makul değildir.

Kitaptaki tanımlardan biri şöyledir: İşletim sistemi, satıcının "işletim sistemi" diye gönderdiği her şeydir. Ama bu sınır da zamanla kayar. Hoca bunu Microsoft örneğiyle anlattı. 1990'lı yıllarda Microsoft işletim sisteminin içinde Internet Explorer geliyordu. O dönemde meşhur başka bir tarayıcı daha vardı: Netscape. Microsoft kendi tarayıcısını işletim sistemine dahil ettiği için ABD'de tekel oluşturduğu gerekçesiyle dava açıldı. Bugün ise bilgisayar yalnız masadaki PC değildir. Cep telefonu da bu işin parçasıdır ve tarayıcısı olmayan bir cep telefonu düşünülemez. Yani sistem programları ile uygulama programları arasındaki sınır zaman içinde bulanıklaşabilir.

> **Ders sonrası not:** Hoca, Microsoft'un davayı kaybedip milyonlarca dolar ceza ödediğini söyledi. ABD'deki dava (United States v. Microsoft) büyük olasılıkla 2001'de bir uzlaşmayla sonuçlandı ve bu davada büyük bir para cezası yoktu. Microsoft'a verilen büyük para cezaları büyük olasılıkla Avrupa Birliği'nden geldi, ör. 2004'te Windows Media Player'ın Windows'a gömülmesi yüzünden verilen yaklaşık 497 milyon avroluk ceza.

5.3 Çekirdek

İşletim sistemi, katman grafiğinde bilgisayar donanımının hemen üstünde görünür. Donanımı sarmalayan bir yapıdır. Hiçbir uygulama işletim sisteminden geçmeden donanıma erişmemelidir. Donanıma bütün erişimi işletim sistemi denetlemelidir. Hocanın benzetmesiyle işletim sistemi donanımı çepeçevre, sert bir kabuk gibi sarar. İşletim sisteminin bu en temel kısmına **çekirdek (kernel)** denir.

> **Hocanın vurgusu:** Donanıma her erişim işletim sisteminden geçmelidir. Hiçbir uygulama doğrudan donanıma erişmemelidir.

5.4 Sistem programları ve uygulama programları

Çekirdek dışındaki parçalar ya bir sistem programıdır ya da bir uygulama programıdır. Uygulama programı işletim sistemiyle birlikte gelebilir ya da sonradan yüklenebilir.

İşletim sisteminin sürmesi ve kullanıcı uygulamalarına servis verebilmesi için başka programlara da ihtiyaç vardır. Bunların hepsini işletim sisteminin içine gömmek her zaman etkin değildir. Bunları ayrı birer program olarak hazırlamak daha iyidir. Örneğin bir işletim sistemi üzerinde uygulama geliştirmek için en basitinden bir metin düzenleyiciye, bir derleyiciye (compiler) ve bir assembler'a ihtiyaç vardır. Bunlar **sistem programlarıdır**. Sisteme eklenti yapmayı, sistemde hata ayıklamayı (debug) ya da diğer uygulamaların servis almasını sağlayan servisler (daemon) de bu gruptadır. Kullanıcıların iş yapması ve iş üretmesi için olan programlar ise **uygulama programlarıdır**.

6. Önyükleme (Bootstrap)

Bilgisayarın güç düğmesine basıldığı andan itibaren sistem bir dizi adımdan geçer. Bilgisayarın başlayıp çalışabilmesi için önce donanımın ilklendirilmesi gerekir. Bu işe bootstrap denir.

1. **Önyükleme programı (bootstrap program):** Üretici tarafından önceden ROM ya da EEPROM üzerine konmuş bir programdır. Güç verildiği andan itibaren devreye girer. Sistemdeki işlemciyi, belleği ve aygıt denetleyicilerini ilklendirir ve çalışmaya hazır hale ge-

tirir. PC dünyasında bu işi BIOS (Basic Input Output System) yapar. Ardından çekirdeğin başlatılması için gerekli ortamı hazırlar.
2. **Önyükleyici (bootloader):** Çekirdeğin belleğe kopyalanması ve çalışmaya başlatılması gerekir. Bunu işletim sisteminin bir parçası olan önyükleyici yapar.

Yani önyükleme işlemi birkaç kademeli bir iş olarak düşünülebilir.

7. Aygıt Denetleyicileri ve Kesmeler

7.1 Denetleyicilerden önce ve sonra

Bir bilgisayar sisteminde bir işlemci, bir bellek, G/Ç birimleri ve bu birimlere ait denetleyiciler (controller) vardır. Yaklaşık 30 yıl önce bu denetleyiciler doğrudan yoktu. Onların görevini de CPU (Central Processing Unit) yerine getiriyordu. Diske ulaşılması gerekiyorsa CPU disk üzerinde gerekli işlemi yapıyor, bilgiyi alıyor ve belleğe kopyalıyordu. USB'den, klavyeden, fareden ya da CD-ROM'dan okuma da işlemci tarafından yapılıyordu. Ekran kartında da bugünkü gibi ayrı bir işlemci yoktu.

Bugün her birimin ya da her veri yolunun (bus) ayrı bir denetleyicisi vardır. Amaç işleri devretmektir (delegate). Diskten okuma yapılacaksa denetleyiciye okunacak adres ve okunacak bilgi miktarı söylenir. Denetleyici diskin ilgili noktasından bilgiyi alır ve kendi yerel belleğine, yani **yerel tamponuna (local buffer)** koyar. Bilgi daha sonra oradan ana belleğe kopyalanır. İşlemci ve diğer çevre birimlerinin hepsi belleğe erişir, ama hepsi ortak bir veri yolu üzerinden erişir.

Akış yöne göre değişir:

- Klavye, fare ya da diskten okuma yapılıyorsa okunan değer denetleyicinin yerel tamponuna konur, CPU da bu bilgiyi alıp belleğe aktarır.
- Yazıcı gibi bir çıkış aygıtında ise ana bellekten denetleyicinin yerel tamponuna konan bilgi aygıta gönderilir.

Denetleyici kendi işini yaparken CPU'nun boş beklemesinin bir anlamı yoktur. Bu süre içinde CPU başka bir sürece ya da başka bir işe yönlendirilebilir. Peki CPU bu işin bittiğini nasıl anlar? Denetleyicinin ona bir **kesme (interrupt)** göndermesi gerekir.

7.2 Kesme nedir?

İşlemci belli bir bellek adresinden başlayarak komutları sırayla çalıştırır. 0. adresten başlar, sonra 4, 8, 12, 16, 20. adreslere geçer. Bu düz çalışma yapısının değiştirilmesi, yani çalışmanın durdurulup farklı bir şeyin çalıştırılması gerekir. Kesme bunu yapan mekanizmadır. Kesme oluştuğunda işlemci o bellek bölgesinden çıkar ve bambaşka bir bellek bölgesindeki kodu çalıştırır.

Hangi kodun çalışacağı, kesmeyi kimin oluşturduğuna göre belirlenir. Kesmeler sistemdeki farklı aygıtlar tarafından oluşturulabilir. Bunun için bir arama tablosu (look-up table) gibi düşünülebilecek bir yapı vardır. Hangi aygıt kesme oluşturduysa tablodaki karşılığına gidilir. Bu tabloya **kesme vektörü (interrupt vector)** denir. Kesme vektörü, mevcut kesme servis yordamlarının hepsinin adreslerinin saklandığı bir tablodur.

7.3 Donanım ve yazılım kesmeleri

Kesme donanımsal olarak oluşabileceği gibi yazılımsal olarak da oluşabilir.

- **Donanım kesmesi:** Bir aygıt denetleyicisinden gelir. Fiziksel olarak işlemcinin bir giriş ucuna bir sinyal gönderilir ve "çalışmayı durdur" denir.
- **Yazılım kesmesi (trap, exception):** İki durumda oluşur.
  - Kullanıcı uygulaması işletim sisteminden bir servis istediğinde. Örneğin G/Ç'yi işletim sistemi gerçekleştirmelidir. G/Ç yapmak isteyen uygulama bunun için tanımlanmış kesmeyi çağırır.
  - Çalıştırılan kodun yaptığı bir hata durumunda. Örneğin bir sayının sıfıra bölünmesiyle oluşan sıfıra bölme (division by zero), çalışma sırasında oluşan bir hatadır ve bir kesme oluşturur.

7.4 Modern işletim sistemi kesmeyle sürülür

İşletim sisteminin farklı kullanıcı uygulamaları arasında geçiş yapabilmesi ve herhangi bir uygulamanın sistemi ele geçirmesini engelleyebilmesi için kesmeyle sürülmesi (interrupt driven) gerekir. Yani işlemcinin çalışmasını kesintiye uğratabilmelidir. Geçmişte kesme kullanmayan işletim sistemleri vardı, ama bunlar artık arkaik sayılır.

> **Hocanın vurgusu:** Kesme olmadan modern bir işletim sisteminden söz edemeyiz. Modern bir işletim sistemi kesmeleri desteklemeli ve kesmeler üzerinden çalışmalıdır.

7.5 Kesme ne zaman denetlenir?

İşlemci, kesme sinyali geldi diye çalışmayı herhangi bir anda bırakmaz. Hoca burada sınıfa bilgisayar donanımı dersinden bir soru sordu: Bir komutun çalıştırılması için hangi üç temel adım vardır?

- **Getir (fetch):** Komut bellekten çekilir.
- **Çöz (decode):** Komutun ne olduğu, işlecinin (operator) ve işlenenlerinin (operand) ne olduğu çözülür. Örneğin anlık (immediate) bir değer mi var, yoksa işlenen bellekten mi getirilecek?
- **Yürüt (execute):** Komut çalıştırılır.

Bu adımların herhangi birinde bir kesme oluşabilir. Ama işlemci işi yarıda bırakıp gitmez. Kesme olup olmadığını komut döngüleri arasında denetler. Getir, çöz, yürüt adımları bölünmeden (atomik olarak) tamamlanır, sonra kesme devreye alınır.

> **Hocanın vurgusu:** İşlemci kesmeyi bir komutun ortasında değil, komut döngüsü (instruction cycle) tamamlandıktan sonra, iki komut arasında denetler.

7.6 Kesme işlenirken durumun saklanması

Bu komut seviyesinden süreç seviyesine çıkıldığında şu görülür: Bir süreç çalışmaya başladığında bazı değerler üretir ve yazmaç (register) setinde değişikliklere yol açar. Çalışma kesintiye uğrayacağı zaman bu durumun saklanması gerekir. Önce en son nerede kalındığı bilgisi bir yere konur. Ayrıca yazmaç setinde çalışmayı etkileyecek değerler vardır, bunlar da bir yere kaydedilir.

Kesme oluşunca yapılanlar sırasıyla şöyledir:

1. Kalınan yer (program sayacı) ve yazmaç değerleri saklanır.
2. Kesme vektörü üzerinden yapılması gereken işe gidilir. Örneğin 5 numaralı kesme geldiyse tablodaki karşılığına gidilir.
3. Kesme tek bir kaynaktan geliyorsa o kaynakla ilgili bilgiye bakılır. Birden fazla aygıta bakan bir G/Ç denetleyicisinden geldiyse, kesmenin tam olarak hangi aygıttan geldiği denetlenir.
4. Okuma, yazma gibi gerekli iş yapılır.
5. Kenara kaydedilen her şey geri yüklenir. Böylece kesilen program kaldığı yerden aynen devam eder.

8. Kesmenin Zaman Çizelgesi ve G/Ç İçin Sistem Çağrısı

8.1 Slayttaki zaman çizelgesi

Slaytta kesmenin zaman içinde nasıl devreye girdiğini gösteren bir çizelge vardır. Üstte kullanıcı sürecinin çalışması (user process executing) ve G/Ç kesme işleme (I/O interrupt processing) çizgisi bulunur. Altta da G/Ç aygıtının boşta mı (idle) yoksa veri aktarıyor mu (transferring) olduğunu gösteren çizgi vardır. Hoca bu grafiğin çıktı (output) üreten bir sisteme ait olduğunu belirtti.

Akış şöyledir:

1. Kullanıcı süreci çalışırken bir G/Ç isteği oluşturulur. G/Ç aygıtı veri aktarma (transferring) durumuna geçer. Çünkü bilginin bellekten alınıp aygıtın yerel tamponuna aktarılması ve oradan aygıta gönderilmesi gerekir.
2. Aktarım bitince G/Ç aygıtı boşta (idle) durumuna geçer. Aynı anda bir kesme oluşur.
3. Kesme, kullanıcı sürecini çalışma durumundan çıkarır. Grafikte kullanıcı süreci çizgisindeki çukur budur. Bu sırada CPU kesme neyle ilgiliyse onu yapar. Giriş ise giriş, çıkış ise çıkış işlemi yapılır. Kesme bir zamanlayıcı kesmesi (timer interrupt) de olabilir, o zaman farklı bir yaklaşım gerekir. G/Ç kesmesinde CPU büyük olasılıkla yerel tampon ile bellek arasında bir şeyler kopyalar.
4. Kesme işlenince CPU tekrar kullanıcı sürecini çalıştırmaya döner.

8.2 Denetleyici CPU'yu serbest bırakır

Eski yapıda bir G/Ç başladığında onun gerçekleştirilmesinden CPU sorumluydu. Aygıt başına ya da veri yolu başına bir denetleyici konunca işlemci serbest kalır. Örneğin sabit diskte ilgili silindire ve sektöre gidip bilgiyi almak, tampona getirmek gibi işlerle işlemci uğraşmaz. Bunları denetleyici yapar. Bu sırada CPU başka bir iş için kullanılabilir.

8.3 G/Ç için sistem çağrısı

Bu düzeni işletim sistemi kurar. Kullanıcı uygulaması G/Ç yapacağı zaman bir **sistem çağrısı (system call)** başlatır. Bu çağrıyla işletim sistemine "Şurada bir G/Ç gerçekleştireceğim, okuma ya da yazma yapacağım, parametreleri de bunlar" der. Okuma yapılacaksa nereden itibaren ve ne kadar okunacağı, yazma yapılacaksa nereden nereye yazılacağı bilgileri parametre olarak geçirilir.

1. Sistem çağrısı bir kesme oluşturur. Kontrol uygulamadan çıkar.
2. Uygulama kenara konur. Kesme vektöründen gerekli yere bakılır ve işletim sistemine geçilir.
3. İşletim sistemi G/Ç denetleyicisini programlar.
4. İsteği başlatan süreç, geri dönüldüğünde çalışamaz. Çalışacak bir şeyi yoktur, G/Ç'nin bitmesini beklemesi gerekir.
5. Bu sırada CPU başka bir uygulamayı çalıştırabilir. Bunun nasıl olduğu ileride ayrıntılı işlenecek.

9. Depolama Yapısı ve Hiyerarşisi

9.1 İşlemciye yakın bellekler

İşlemcinin üzerinde çok temel bellekler vardır. Hoca "bellekler" dedi, çünkü farklı tiplerdedir.

- **Yazmaç (register):** Bir işlemin gerçekleşmesi için işlemcinin veriye ihtiyacı vardır. İşlemcinin veriye doğrudan ulaşabildiği yer yazmaçtır.
- **Önbellek (cache):** İşlemcinin içinde başka bir bellek alanı daha vardır. Tek seviyeli olmak zorunda değildir. L1, L2, L3 diye giden bir silsile vardır.
- **Ana bellek (main memory):** Birincil bellek (primary memory) ya da birincil depolama (primary storage) diye de geçer.
- **İkincil depolama (secondary storage):** Sabit disk ve SSD gibi.
- Bunların altında piramit biçiminde aşağı doğru giden farklı saklama alanları vardır.

9.2 Ana bellek: rastgele erişimli ve uçucu

Ana bellek yapısı gereği **rastgele erişimlidir (random access)**. İstenen adrese doğrudan erişilebilir. Belli bir adresten başlayıp istenen adrese gelene kadar her şeyi sırayla okumak gerekmez. Bu yüzden RAM (random access memory) denir.

Ana belleğin bir diğer özelliği **uçucu (volatile)** olmasıdır. Yazılan bilgiyi ancak elektrik olduğu sürece tutar. Bilgisayarın fişi çekildiği an bellekteki her şey gider. Bilgisayar açıldığında bellek çöp (garbage) içerir, içinde her şey olabilir.

- **Hazırda bekletme (hibernate):** Bellek içeriği doğrudan korunmaz. Bellek içeriği kenara kaydedilir, açılışta geri yüklenir.

- Duruma göre belleğin elektrik bağlantısı kesilmez, böylece bilgi kaybolmaz.
- **Pil desteği (battery backup):** Mainframe gibi büyük sunucu sistemlerinde RAM'lerin pil desteği vardır. Elektrik kesilince pil devreye girer, sistem çalışmasını tamamlar ve güvenli bir duruma (safe state) geçer. Ama bu belleği kalıcı (non-volatile) yapmaz. Bellek yine uçucudur, yalnız arkada pil olduğu sürece bilgi tutar.

9.3 İkincil depolama: kalıcı

Sabit diskte bilgi metal ya da cam diskler üzerinde saklanır. Hoca bunu "taş tablete kazır gibi" diye benzetti. Bilgi elektrik bağlantısı olmasa da uzun süre saklanabilir. Bu yüzden sabit disk **kalıcıdır (non-volatile)**, uçuculuğu yoktur.

> **Ders sonrası not:** Hoca saklamayı "metalde ferit çubukların şartlandırılması" diye anlattı. Sabit diskte veri büyük olasılıkla plaka yüzeyindeki manyetik malzemenin küçük bölgelerinin mıknatıslanma yönüyle saklanır. Kalıcılık bu manyetik durumdan gelir.

Bugün katı hal sürücüsü (solid state drive, SSD) olmayan dizüstü bilgisayar herhalde yok ya da çok azdır. SSD'ler farklı yapılarda üretilir, ör. NAND yapısında. Bunlar da ikincil depolamadır, ama sabit diskten çok daha hızlıdır.

9.4 Depolama piramidi

Bu farklara göre bir piramit ortaya çıkar. Yukarıdan aşağı:

1. Yazmaçlar
2. Önbellek
3. Ana bellek
4. Katı hal diski (SSD)
5. Sabit disk
6. Optik disk
7. Manyetik bant

Piramidin en üstünde en hızlı bellek vardır. Yukarı çıkıldıkça maliyet artar, maliyet artınca boyut küçülür. Aşağı inildikçe hız düşer, fiyat azalır, kapasite artar. Bu yüzden yazmaç az sayıdadır, çok hızlıdır ve etkin kullanılmalıdır. Önbellek de az ve hızlıdır, etkin kullanılmalıdır. Ana bellek de hızlıdır ama kalıcı değildir ve ikincil depolamaya göre pahalıdır, dolayısıyla o da etkin kullanılmalıdır.

Manyetik bant (magnetic tape): Piramidin en altındadır ve yedekleme (backup) için kullanılır. Erişimi sıralıdır (sequential access). Hoca bunu eski araba teyplerindeki kasete benzetti. Bir bant kartuşunun kapasitesi 4, 8, 16, 32 terabayt gibi değerlerdedir. Banda bilgi yazmak ya da banttaki herhangi bir bilgiyi çıkarmak kolay değildir.

10. Önbelleğe Alma ve Tutarlılık Sorunu

10.1 Önbelleğe alma (caching)

Piramitteki her basamak, bir alttaki basamak için önbellek gibi davranabilir. Örneğin ana bellekte çok sık ulaşılan bir bilgi varsa, ondan daha hızlı olan önbelleğe çekilir. Her seferinde ana belleğe gitmek yerine önbellekten daha hızlı ulaşılır. Benzer şekilde SSD ya da sabit diskte çok sık ulaşılan bir bilgi, bir üst kademe olan ana belleğe aktarılabilir. Daha hızlı olan üst seviyeyi, daha yavaş olan alt seviye için bir ara geçiş ve hızlandırma noktası olarak kullanmaya **önbelleğe alma (caching)** denir.

10.2 Önbelleğe almaya kim karar verir?

Bir öğrenci önbelleğe alma kararının kim tarafından verildiğini sordu. Hoca cevabın hangi seviyede konuşulduğuna bağlı olduğunu söyledi:

- **Donanım:** Ana bellekten CPU önbelleğine taşıma donanımın denetimindedir. Donanım bellek erişimlerini izler, çok erişilen bölgeyi önbelleğe çeker. Önbellek ile yazmaç arasındaki geçişi de donanım yönetir.

- **İşletim sistemi:** Çoklu programlamada çalışacak süreçlerin belleğe alınması, hatta tamamı bellekte olmadan bir kısmı bellekte tutularak çalıştırılması işletim sisteminin yönettiği bir iştir.
- **Uygulama yazılımı:** Bant havuzu ile disk havuzu arasındaki taşımayı işletim sistemi yapmaz. Bir uygulama yazılımı yapar. Bant havuzunun (tape pool) yanında sabit disk ya da optik disk üzerinde de bir havuz tanımlanır. Örneğin bir petabaytlık disk yedekleme havuzu olarak tanımlanır, arkada da birkaç petabaytlık bir bant havuzu vardır. Disk havuzu %75'in üstüne çıkınca veri banda aktarılır. %75 doluluğa ulaşana kadar disk alanı kullanılır, hızlı yazılır ve okunur. Bu da bir tür önbelleğe almadır. Ama CPU önbelleği ile ana bellek arasındaki ya da sabit disk ile ana bellek arasındaki kadar hızlı bir geçiş değildir.

  Hocanın vurgusu: Önbelleğe alma kararı, hangi seviyede yapıldığına bağlı olarak donanım, işletim sistemi ya da uygulama yazılımı tarafından verilir.

10.3 Tutarlılık (consistency) sorunu

Önbelleğe almanın bir sorunu vardır. Bu sorun özellikle çok işlemcili sistemlerde ve çok çekirdekli işlemcilerde ortaya çıkar. Daha yavaş bir bellekten bilgi alınıp daha hızlı bir yere konduğunda, o bilgi üzerinde bir değişiklik yapılırsa **tutarsızlık (inconsistency)** başlar.

Örnek: Bellekteki bir bilgi alınıp önbelleğe konur ve önbellekte değiştirilir. Sistemde birden fazla işlemci vardır ve diğer işlemcideki bir süreç aynı yere ulaşmaya çalışır. Birinci işlemcinin önbelleğindeki bilgi günceldir, ana bellekteki ise güncel değildir. İkinci süreç ulaştığında tutarsız veri okur. Bu yüzden değişiklik olduğunda hemen belleğe yansıması ve yansıyana kadar kimsenin oraya erişmemesi gerekir. Burada kilitler (lock) devreye girer.

> **Ek bilgi (derste anlatılmadı):** Çok çekirdekli işlemcilerde bu sorun büyük olasılıkla donanımda önbellek tutarlılığı (cache coherence) protokolleriyle çözülür. Yaygın bir örnek MESI protokolüdür: Bir çekirdek bir önbellek satırını değiştirdiğinde diğer çekirdeklerdeki kopyalar geçersiz (invalid) işaretlenir. Kilitler ise yazılım seviyesinde, birden fazla adımlık işlemlerin bölünmeden yapılmasını sağlar.

11. Doğrudan Bellek Erişimi (DMA) ve von Neumann / Harvard Mimarileri

11.1 DMA

Şimdiye kadar anlatılan düzende G/Ç'yi denetleyici yönetir ve bitince haber verir. Ama okuma yapıldıysa yerel tampondaki bilgiyi alıp belleğe kopyalayan yine CPU'dur. Bu hâlâ çok etkin değildir. CPU her seferinde işini bırakıp bununla uğraşır. Oysa o süre içinde CPU başka bir şey için kullanılabilir.

Bunun çözümü **doğrudan bellek erişimi (Direct Memory Access, DMA)** modülüdür. DMA denetleyicisi programlanır: "Bu aygıtın yerel tamponundan ana belleğin şu bölgesine bilgiyi kopyala" ya da "ana bellekten şu bölgeyi al, şu aygıtın şurasına kopyala" denir. Kopyalamayı CPU yerine DMA denetleyicisi yapar. Böylece CPU bir adım daha serbest kalır.

> **Ek bilgi (derste anlatılmadı):** DMA'da denetleyici büyük olasılıkla bütün bir veri bloğunu bellek ile aygıt arasında aktarır ve CPU'ya bayt başına değil, blok bittiğinde tek bir kesme gönderir. Bu sayede yüksek hızlı aygıtlarda CPU üzerindeki kesme yükü çok azalır.

11.2 von Neumann ve Harvard mimarileri

Slayttaki şemada bir tarafta CPU, bir tarafta bellek, bir tarafta aygıt vardır. Aygıttan okunan bilgi yerel tampona alınır, CPU ile aygıt arasındaki veri hattından alınıp ana belleğe konur. Hoca şuna dikkat çekti: Ana bellek ile CPU arasında hem komut yürütme döngüsü (instruction execution cycle) hem de veri hareketi (data movement) vardır. Hoca bu mimariyi "tek bir veri yolu üzerinden hem verinin hem de adres bilgisinin aktarılması" diye tanımladı ve buna **von Neumann mimarisi** dendiğini söyledi.

Hoca **Harvard mimarisini** ise "veri ve adres için ayrı veri yolları" diye anlattı. Avantajı daha hızlı olmasıdır: von Neumann'da önce adres söylenir, sonra veri, Harvard'da ikisi aynı anda.

Bunun karşılığı ise maliyettir. Bu yüzden günümüzde sunucu sistemlerinin büyük çoğunluğu ve kişisel bilgisayarlar von Neumann mimarisi üzerine kuruludur. Harvard mimarisini kullanan sistemler de vardır, özellikle mikrodenetleyicilerde çok kullanılır.

> **Ders sonrası not:** Hoca iki mimariyi adres ve veri ayrımı üzerinden anlattı; doğru ayrım büyük olasılıkla komut ile veri arasındadır. von Neumann mimarisinde komutlar ve veri aynı bellekte durur ve aynı yoldan taşınır. Slayttaki şemada da bellek kutusunda "instructions and data" yazar. Harvard mimarisinde komut belleği ile veri belleği ve bunlara giden yollar ayrıdır. Bu yüzden komut getirme ile veri okuma aynı anda yapılabilir.

12. Çok İşlemcili Sistemler

12.1 Tanım ve avantajlar

Şimdiye kadar hep tek bir işlemciden söz edildi. Burada işlemci, genel amaçlı (general purpose), yani birçok farklı işi yapabilen bir işlemcidir. Tek işlemcili bir sistemde bile belli bir işe ayrılmış (dedicated) özel işlemciler olabilir. Bu sistem yine tek işlemcili (single CPU) sayılır. Çok işlemcili (multiprocessor) sistem denince içinde birden fazla genel amaçlı işlemci olması gerekir.

Çok işlemcili sistemlere neden ihtiyaç vardır?

- **Daha fazla iş:** Birim zamanda daha çok işlem yapılır.
- **Ölçek ekonomisi:** Sistemi ölçeklemenin iki yolu vardır: bir bilgisayar yerine iki bilgisayar koymak ya da aynı bilgisayara iki işlemci koymak. İkincisi daha ucuzdur, çünkü işlemciler G/Ç aygıtlarını ve diğer her şeyi paylaşır.
- **Güvenilirlik (reliability):** Tek işlemcili mimaride işlemcinin başına bir şey gelirse sistem çalışamaz. İki, dört ya da sekiz işlemcili bir sistemde bir işlemci paketi bozulursa diğerleri sistemi çalıştırmaya devam eder. Sunucu sistemiyse verilen servis biraz düşer, ama kesilmez.

12.2 Asimetrik ve simetrik çoklu işlem

Çok işlemcili sistem kurmanın birden fazla yolu vardır.

- **Simetrik çoklu işlem (Symmetric Multiprocessing, SMP):** Bütün işlemciler aynı seviyededir ve aynı işleri yapabilir. İşlemciler arasında efendi-köle, yönetici-işçi (master-worker) ya da birincil-ikincil (primary-secondary) gibi bir seviyelendirme yoktur. Herkes eşittir, herhangi bir iş herhangi bir işlemcide çalışabilir. Günümüzde kullanılan sistemlerin hepsi SMP'dir.
- **Asimetrik çoklu işlem:** İşlemcilerden biri ya da birkaçı belli bir işe konumlandırılmıştır. Biri işi yönetir: Diğer işlemcilerde neyin, ne zaman, nasıl çalışacağını denetler. Diğerleri genel işleri yapar. Hoca, CPU'nun yanında güçlü bir ekran kartı (GPU) varsa onun da derin öğrenme gibi belli bir iş için işleme dahil edilmesini asimetriye örnek verdi. GPU genel uygulama yazılımları için değil, belli bir iş için kullanılır, yani işlemciler aynı işleri yapmaz.

  > **Ders sonrası not:** Ders kitabındaki asimetrik çoklu işlem büyük olasılıkla genel amaçlı işlemciler arasındaki yönetici-işçi ilişkisidir. CPU yanında GPU kullanmak ise farklı tür işlemcilerin birlikte kullanıldığı heterojen hesaplamadır. GPU genel amaçlı bir CPU olmadığı için hocanın örneği bir benzetme olarak düşünülmelidir.

12.3 Sıkı ve gevşek bağlı sistemler

Bir kutunun içinde birden fazla işlemcinin bulunduğu yapılara **sıkı bağlı (tightly coupled)** sistem denir. Çok işlemcili sistem birbirinden ayrı bilgisayarların bir araya getirilmesiyle kurulsaydı buna **gevşek bağlı (loosely coupled)** sistem denirdi. Kümeler (cluster) bunun bir örneğidir.

12.4 SMP'de bellek erişimi

SMP yapısında sistemde birden fazla işlemci vardır ve her birinin kendine özel yazmaçları ve önbelleği vardır. Hepsi ortak ana belleğe erişir. Ama bu erişimin bir önceliğe göre düzenlenmesi gerekir. İki işlemci aynı anda erişmek isterse bir sıraya konmalıdır. Hoca bunun genelde işlemci numarası üzerinden yapıldığını, numarası en düşük olanın en yüksek önceliğe sahip olduğunu söyledi. Önceki bölümdeki önbellek tutarlılığı sorunu burada da sık sık karşımıza çıkar.

> **Ders sonrası not:** "Numarası düşük işlemci önce erişir" büyük olasılıkla genel bir kural değildir. Erişim sırası veri yolu hakemliği (bus arbitration) ya da bellek denetleyicisi tarafından belirlenir ve kullanılan yöntem mimariye göre değişir, ör. sıralı dönüşümlü (round-robin) hakemlik.

12.5 Çok çekirdekli tasarım

Çok çekirdekli (multi-core) ya da çift çekirdekli (dual-core) tasarımda yapı daha da karmaşıklaşır. Ana bellek vardır ve her çekirdeğin kendi önbelleği vardır. Diyelim ki CPU 1 belleğin bir bölgesine çok erişti ve donanım o bölgedeki bilgiyi CPU 1'in önbelleğine aktardı. Bu önbellekte bilgi değişirse, CPU 0 ya da CPU 2 o bellek alanına ulaştığında değişmiş bilgiye değil, eski (orijinal) bilgiye erişir. Bu yüzden önbelleklerin senkronize edilmesi ve denetlenmesi gerekir.

Günümüzde daha yaygın olan, bir sisteme birden fazla işlemci paketi koymak yerine bir işlemci paketinin içine birden fazla çekirdek (core) koymaktır. Masaüstü ve dizüstü bilgisayarlarda tek paketin içinde birden fazla çekirdek olan işlemciler kullanılır. Sunucularda ise ikisi bir arada kullanılır: Bir sunucuda genelde birden fazla çekirdeği olan birden fazla işlemci vardır.

Çok çekirdekli tasarımın güzelliği, aynı ya da çok daha dar bir alanda daha çok işlem gücü sağlamasıdır. Bedeli ise birbiriyle ilişkili iki sorundur: alan daraldığı için ısınma ve artan güç tüketimi. Buna karşılık maliyet çok düşer.

12.6 Sanal çekirdekler

Hoca çekirdekler üzerinde bir seviyede sanallaştırma da yapıldığını söyledi. Bir işlemcinin özelliklerinde 4 çekirdek yazar, ama işletim sistemi üzerinden bakıldığında 8 çekirdek görülür. Hoca bunu, işlemcinin alt tarafta çekirdeklerin işlemci zamanlarını zaman paylaşımıyla iki katına çıkarması diye anlattı. Bu doğrudan iki kat başarım vermez, ama kullanım oranında (utilization) bir iyileşme sağlar.

> **Ders sonrası not:** Bu teknik büyük olasılıkla eşzamanlı çoklu iş parçacığıdır (simultaneous multithreading, SMT; Intel'de hyper-threading). Bir fiziksel çekirdek, iki donanım iş parçacığının durumunu (yazmaç setlerini) ayrı tutar ve yürütme birimleri iki iş parçacığı arasında aynı anda paylaşılır. Bir iş parçacığı bellek beklerken diğeri boşta kalan birimleri kullanır. Yani sıradan bir zaman paylaşımı değildir; kazanç da bu yüzden iki kat değildir.

13. Kümelenmiş Sistemler

Birden fazla işlemciyi bir araya getirmenin tek yolu aynı sistemin içine koymak değildir. Bağımsız sistemleri birbirine bağlayıp bunlar üzerinden bir çoklu işlem sistemi kurmak da mümkündür. Buna çok işlemcili sistem denmez, çünkü her biri kendi işlemcisi olan bağımsız bir sistemdir. Ama amaç birden fazla işlemciyi birlikte kullanabilmektir. Birden fazla düğümün (node) bir araya gelerek oluşturduğu bu sisteme **küme (cluster)** denir. Kümenin adı, kurulma amacına göre değişir.

13.1 Yüksek başarımlı hesaplama (HPC)

Amaç işlem gücünü artırmak, birim zamanda yapılan iş miktarını artırmak ya da bir işin süresini kısaltmaksa, yüksek başarımlı hesaplamaya (high performance computing, HPC) gidilir. Yapılan iş parçalara bölünüp paralel çalıştırılmaya çalışılır. Ama her şey paralelleştirilemez. Sistemde paralelleştirilebilen bir kısım ve seri kalması gereken bir kısım vardır. Seri kalması

gereken kısmın büyüklüğü sistemin ne kadar etkin olacağını belirler. Geri kalanın ne kadar paralelleşebildiği de kurulan sistemin ne kadar etkin kullanılabileceğini ortaya koyar.

> **Ek bilgi (derste anlatılmadı):** Bu ilişki büyük olasılıkla Amdahl yasasıyla ifade edilir. İşin paralelleştirilebilen oranı p, işlemci sayısı N ise elde edilebilecek hızlanma $S(N) = \dfrac{1}{(1-p) + p/N}$ olur. N ne kadar büyürse büyüsün hızlanma $1/(1-p)$ ile sınırlıdır; yani seri kısım küçük olsa bile tavanı o belirler.

13.2 Yüksek erişilebilirlik

Kümelerin bir başka amacı yüksek erişilebilirliktir (high availability). Çok işlemcili sistemde bir işlemci bozulunca diğerlerinin çalışmaya devam ettiği söylenmişti, bu da bir erişilebilirlik sağlar. Ama kümede bu çok daha güçlüdür. Örneğin birbirine bağlı 40 düğümün 20'si ayrı ayrı elektrik ya da donanım sorunu yaşasa bile küme çalışmaya devam eder.

Yüksek erişilebilirlik farklı düzenlerle sağlanabilir:

- **Asimetrik kümeleme:** 40 düğümün bir kısmı kenara ayrılıp sıcak beklemede (hot standby) tutulur. Bir sorun olunca biri devreye girer, bir sorun daha olunca bir diğeri. Hocanın benzetmesiyle bu düğümler kenarda bekleyen yedek oyuncular gibidir.
- **Simetrik kümeleme:** Bütün düğümler aynı anda oyunun içindedir. İstek miktarı arttıkça ve azaldıkça yük aralarında paylaşılır.

13.3 Günümüzde dinamik kümeler

Hoca bu işin bugün daha dinamik yapıldığını, bunun bir nedeninin de yeşil hesaplama (green computing) olduğunu söyledi. Kümelerde düğümler yerel depolama yerine genelde arkadaki merkezi bir depolama sistemine, bir depolama alanı ağına (storage area network, SAN) bağlanır. Böylece bu ağdaki herhangi bir servisin imajı alınıp öndeki sunuculardan herhangi birine kopyalanabilir. İstek sayısına ve sistemdeki yüke göre çalışan servis kopyalarının sayısı artırılıp azaltılabilir.

14. Çoklu Programlama ve Zaman Paylaşımı

Hoca bu kavramları dersin sonunda kısaca tanıttı ve sonraki derste tekrar ele alacağını söyledi.

14.1 Çoklu programlama (multiprogramming)

İlk bilgisayar sistemlerinde program delikli kartlara (punch card) delik delinerek yazılırdı. Kartlar bir operatöre verilirdi. Operatör sizin ve başkalarının programlarını sıraya dizer ve sırayla çalıştırırdı. Daha sonra operatör devreden çıktı ve onun yerini işletim sistemi aldı. Arada tek kullanıcının sistemin bütün kaynaklarına sahip olduğu kişisel bilgisayarlar da ortaya çıktı.

Tek kullanıcının bütün kaynaklara sahip olduğu yapıda her kaynağı tam etkin kullanmak söz konusu değildir. Bilgisayarın açıldığı andan kapandığı ana kadar CPU'yu ve G/Ç aygıtlarının hepsini sonuna kadar kullanacak bir iş yoktur. Bu yalnız kişisel bilgisayarlar için değil, sunucular için de geçerlidir. Oysa sistemde zaten çalışması gereken birden fazla iş vardır. CPU, bellek ve G/Ç kullanımını verimli hale getirecek şekilde birden fazla program arasında geçiş yapmak gerekir. **Çoklu programlama** budur.

14.2 Zaman paylaşımı (time sharing, multitasking)

Sistem etkileşimli (interactive) bir sistemse, örneğin masaüstü bilgisayarlar gibi, kullanıcıya çalıştırdığı işin ilerlediği gösterilmelidir. Bir düzenleyicide tuşlara basıp ekranda hiçbir şey görmemek normal ve kabul edilebilir değildir. Tuşa basıldıkça yazının ekranda çıkması istenir. Aynı sırada Word'de yazarken fareyle bir tarayıcı açılırsa tarayıcının çalıştığı görülmek istenir. Tarayıcıda gezinirken arkada açılan müziğin de çalmaya devam etmesi istenir.

Bu, "bu işi başlattım, bitmeden öbürünü açamam" mantığıyla olmaz. Birden fazla işi aynı anda yapılıyormuş gibi yürütmek gerekir. Bunun için **zaman paylaşımı (time sharing)** ya da **çoklu görev (multitasking)** yapılır. Birden fazla iş arasında sürekli geçiş yapılır ve hepsi azar azar

ilerler. Bir insanın algılayabileceği zaman farkı, geçiş yapma süresinden çok daha uzundur. Bu yüzden işler arasında rahatça geçiş yapılır ve hepsi aynı anda çalışıyormuş izlenimi verilir.

14.3 Çoklu programlamada bellek

Bir programın çalıştırılabilmesi için önce ana belleğe alınması gerekir. Bu yüzden çoklu programlama yapan bir sistemin belleğinde işletim sisteminin kendisi ve çalıştırılacak işler bulunur. Kitapta iş (job), süreç (process) ve görev (task) terimleri birbirinin yerine kullanılır.

Bu düzen ileride bir sorun doğurur. Örneğin belleğe 1, 2, 3 ve 4 numaralı işler alınır. Sonra 2 biter ve ortada bir boşluk kalır. Sonra 4 de biter. 5 numaralı iş bir yere konur, örneğin 2'nin boşluğuna sığar, ama 5 ile 3 arasında yine bir boşluk kalır ve oraya hiçbir şey konamaz. Bu sorun bellek yönetimi konusunda işlenecek.

> **Ek bilgi (derste anlatılmadı):** Bu sorunun adı büyük olasılıkla dış parçalanmadır (external fragmentation): Bellekte toplamda yeterli boş alan olduğu halde, bu alan parçalı olduğu için yeni bir işe bitişik bir yer bulunamaz.

Örnek Sorular

Soru 1 (derste sözlü, 01:01:50): Bir komutun çalıştırılabilmesi için hangi üç temel adım vardır? Kesme bu adımların neresinde denetlenir?

> **Çözüm (derste verildi):** Getir (fetch), çöz (decode), yürüt (execute). Komut önce bellekten çekilir, sonra işleci ve işlenenleri çözülür, sonra yürütülür. İşlemci kesmeyi bu adımların ortasında değil, komut döngüsü tamamlandıktan sonra, iki komut arasında denetler.

Soru 2 (derste sözlü, 01:21:35): Önbelleğe alma mekanizmasını kim yönetir? Örneğin RAM'den işlemci önbelleğine veri çekme kararını kim verir?

> **Çözüm (derste verildi):** Seviyeye bağlıdır. Ana bellek ile CPU önbelleği arasındaki ve önbellek ile yazmaç arasındaki taşımayı donanım yönetir. Süreçlerin belleğe alınmasını işletim sistemi yönetir. Disk havuzundan bant havuzuna aktarım gibi alt seviyelerdeki taşımayı bir uygulama yazılımı yapabilir, ör. disk havuzu %75'i geçince verinin banda aktarılması.

Soru 3 (derste sözlü, 01:31:12): Çok işlemcili sistemlere neden ihtiyaç vardır? Bir bilgisayar yerine iki bilgisayar mı koymak, yoksa aynı bilgisayara iki işlemci mi koymak daha ekonomiktir?

> **Çözüm (derste verildi):** Birim zamanda daha çok iş yapılır, ölçekleme daha ekonomiktir ve güvenilirlik artar. Aynı bilgisayara iki işlemci koymak daha ucuzdur, çünkü işlemciler G/Ç aygıtlarını ve diğer bileşenleri paylaşır. Bir işlemci bozulursa diğerleri sistemi çalıştırmaya devam eder.

Soru 4 (derste sözlü, 01:41:28): Yüksek başarımlı bir kümede her işi paralelleştirebilir miyiz?

> **Çözüm (derste verildi):** Hayır. Her işte paralelleştirilebilen bir kısım ve seri kalması gereken bir kısım vardır. Seri kısmın büyüklüğü sistemin ne kadar etkin olacağını belirler.

Dersin Toparlaması

Bu ilk derste önce dersin yapısı, laboratuvar uygulamaları ve kaynak kitaplar tanıtıldı, ardından dönemin yol haritası çizildi. İşletim sistemi, bilgisayar sisteminin dört bileşeninden biri olarak tanımlandı. Kullanıcı bakışından kolay kullanımı sağlayan bir ortam, sistem bakışından ise bir kaynak yöneticisi ve kontrol programıdır. Ne yaptığı, sistemin kişisel bilgisayar, paylaşımlı sunucu ya da mobil cihaz olmasına göre değişir. Donanımı sert bir kabuk gibi saran çekirdek ile sistem ve uygulama programları ayrıldı, önyükleme sırası BIOS'tan önyükleyiciye kadar izlendi.

Dersin merkezinde kesme mekanizması vardı. Aygıt denetleyicileri işi CPU'dan devralır, bitince kesmeyle haber verir. Kesme vektörü doğru servis yordamını bulur, durum saklanıp geri

yüklenir ve işlemci kesmeyi yalnız iki komut arasında denetler. Depolama hiyerarşisi, önbelleğe alma ve tutarlılık sorunu, DMA, von Neumann ve Harvard mimarileri, çok işlemcili ve çok çekirdekli sistemler ile kümeler işlendi. Ders, çoklu programlama ve zaman paylaşımına kısa bir girişle bitti. Hoca bu kavramları ve bellekte oluşan boşluk sorununu sonraki derste kaldığı yerden sürdüreceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.